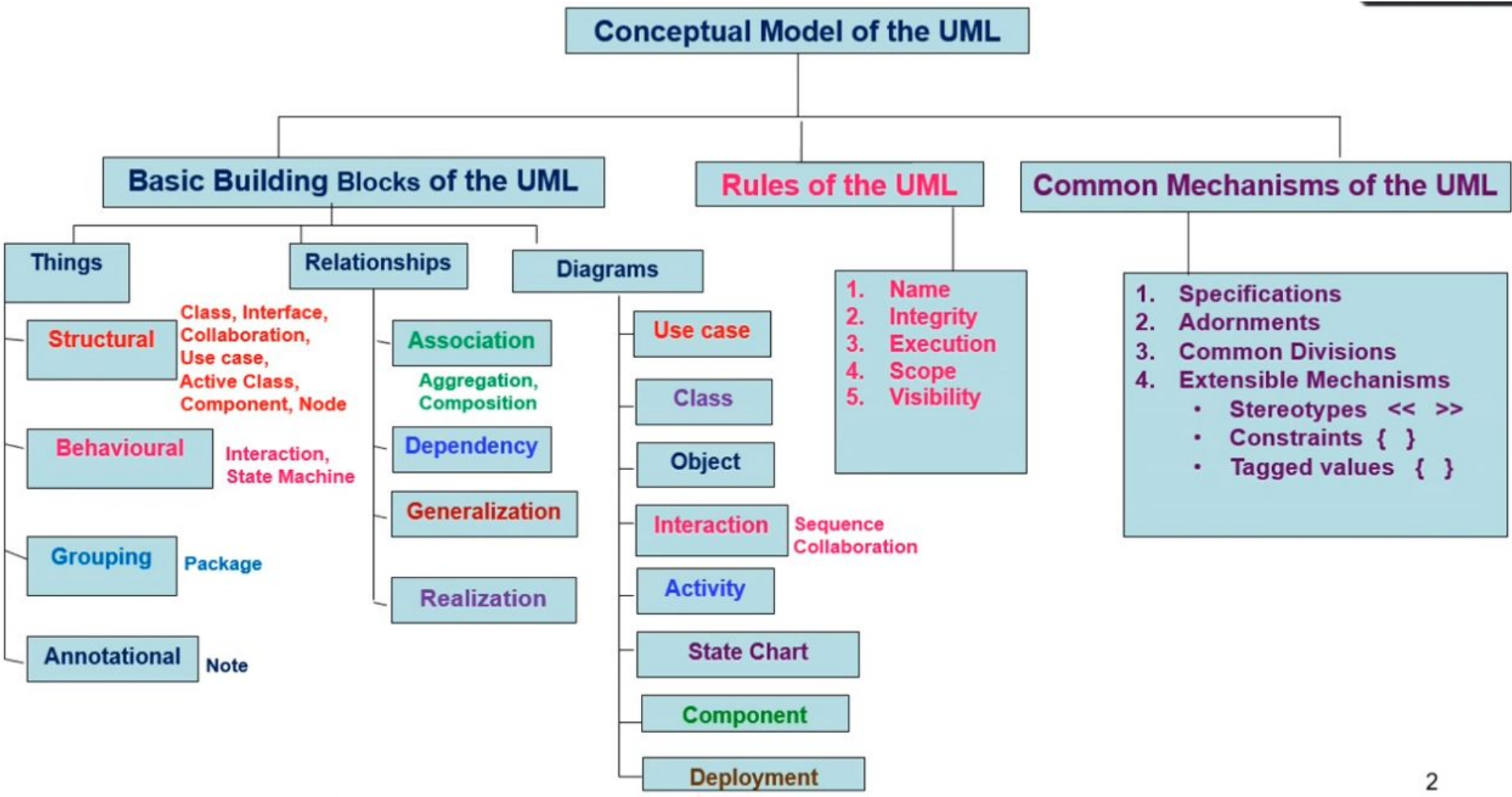
UML

- A Standardized modelling language.
- A way of interaction between different stakeholders of a system
- Supports a broad range of applications
- Independent of particular programming languages and development processes
- Developed for specifying, visualizing, and documenting artifacts of a system.
- It is a pictorial language used to make software blueprints.
- A picture is worth a thousand words, this idiom absolutely fits describing UML.


Conceptual Model of the UML
Basic Building Blocks of the UML
Rules of the UML
Common Mechanisms of the UML
Things
Relationships
Diagrams
Structural
Class, Interface, Collaboration, Use case, Active Class, Component, Node
Behavioural
Interaction, State Machine
Grouping
Package
Annotational
Note
Association
Aggregation, Composition
Dependency
Generalization
Realization
Use case
Class
Object
Interaction
Sequence Collaboration
Activity
State Chart
Component
Deployment
1. Name
2. Integrity
3. Execution
4. Scope
5. Visibility
1. Specifications
2. Adornments
3. Common Divisions
4. Extensible Mechanisms
• Stereotypes << >>
• Constraints { }
• Tagged values { }

Structural things

- Defines the static part of the model. They represent the physical and conceptual elements.

1. **Class** - An object with defined attributes and operations. Class represents a set of objects having similar responsibilities.

2. **Interface** - Interface defines a set of operations, which specify the responsibility of a class. Externally visible behavior of that class.

3. **Collaboration** -Collaboration defines an interaction between elements.

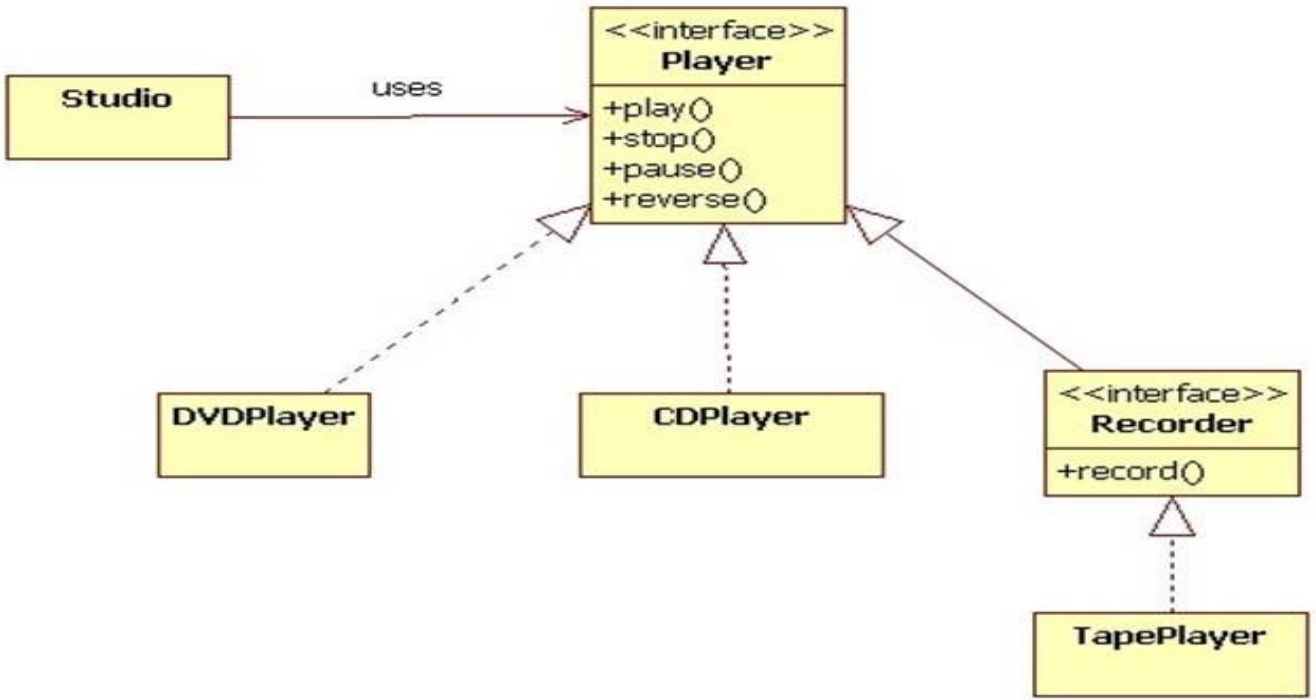
4. **Use case** -Use case represents a set of actions performed by a system for a specific goal.

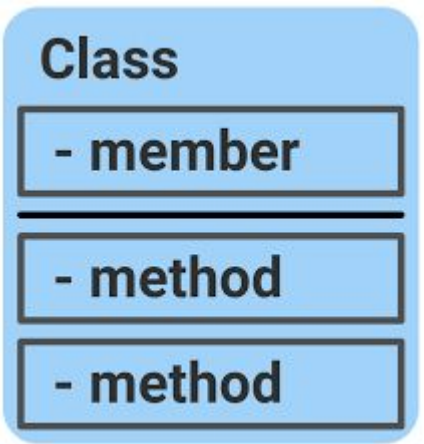
5. **Component** -Component describes the physical part of a system. A physical and replaceable part of a system that implements a number of interfaces.

6. **Node** - A node can be defined as a physical element that exists at run time and represents a resource.

Class Diagram

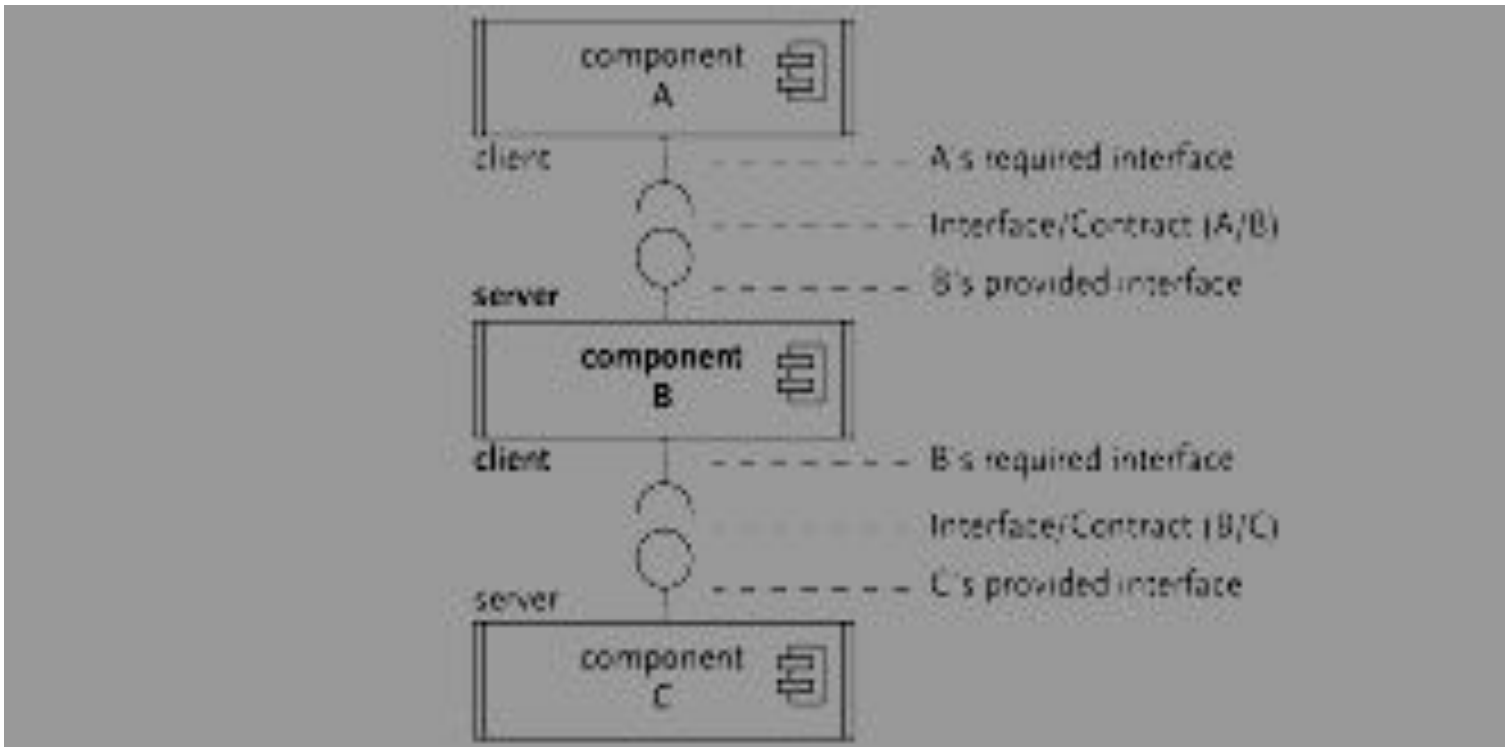
Classes are used to represent objects. Objects can be anything having properties and responsibility.





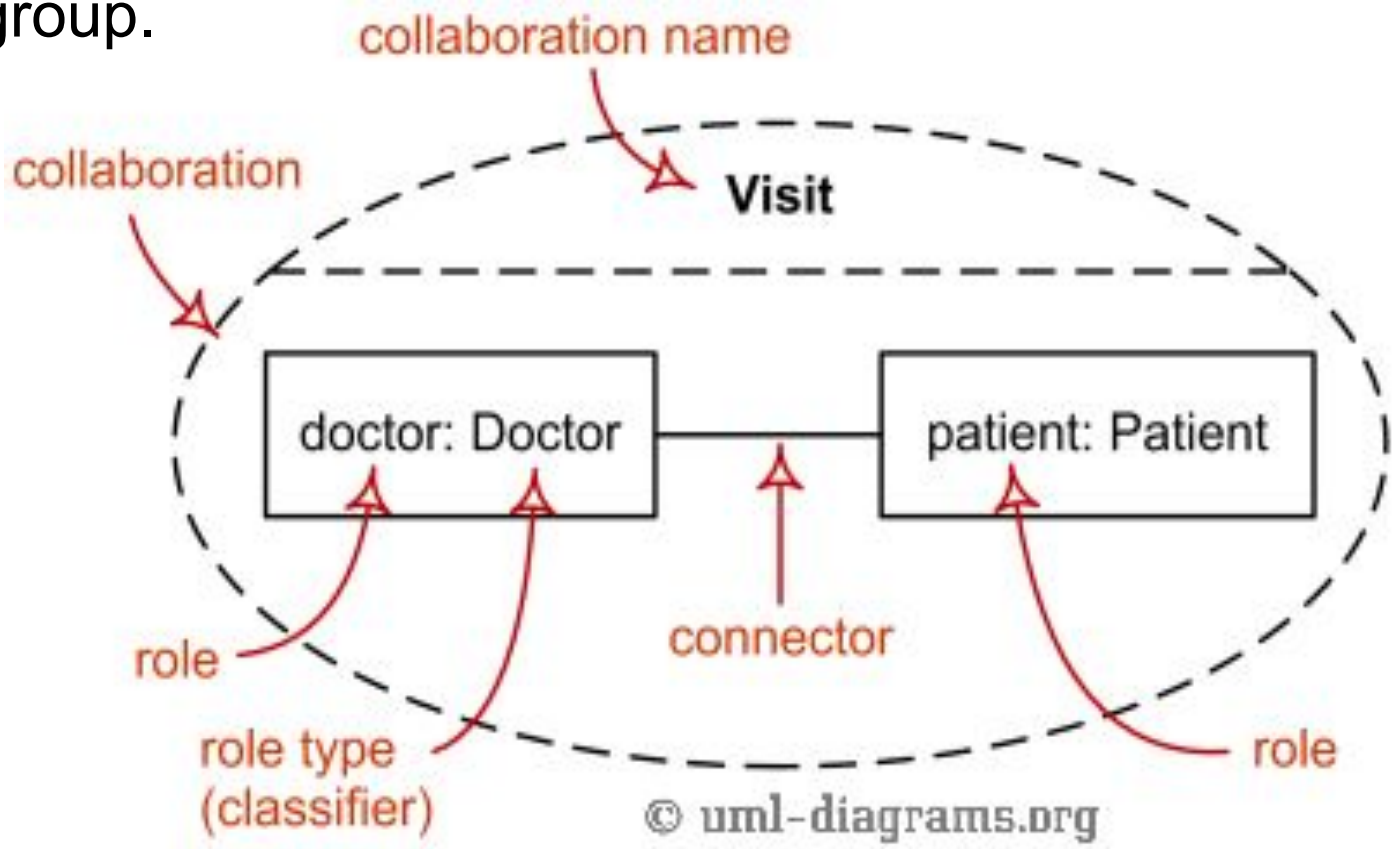
Interface

Interface is used to describe the functionality without implementation. Interface is just like a template where you define different functions, not the implementation.



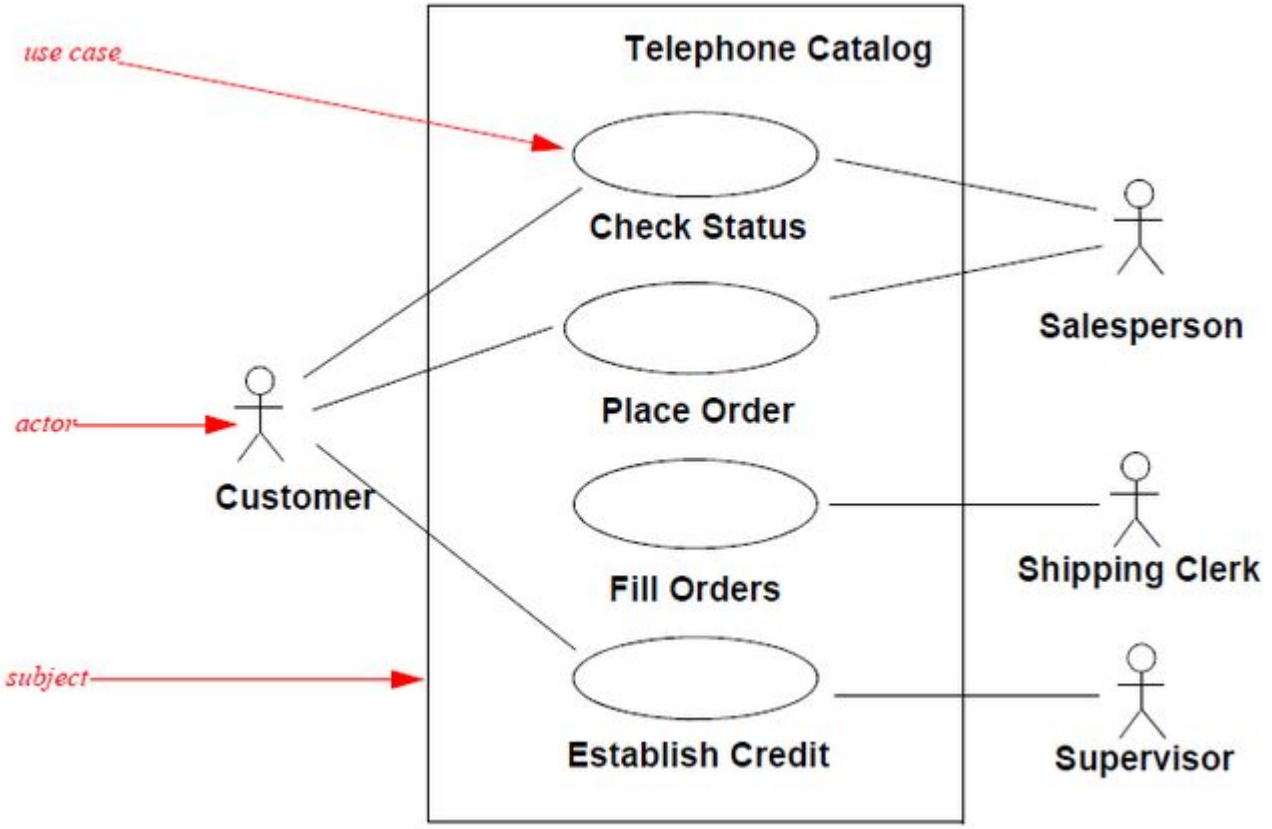
collaboration

Collaboration represents responsibilities. Generally, responsibilities are in a group.



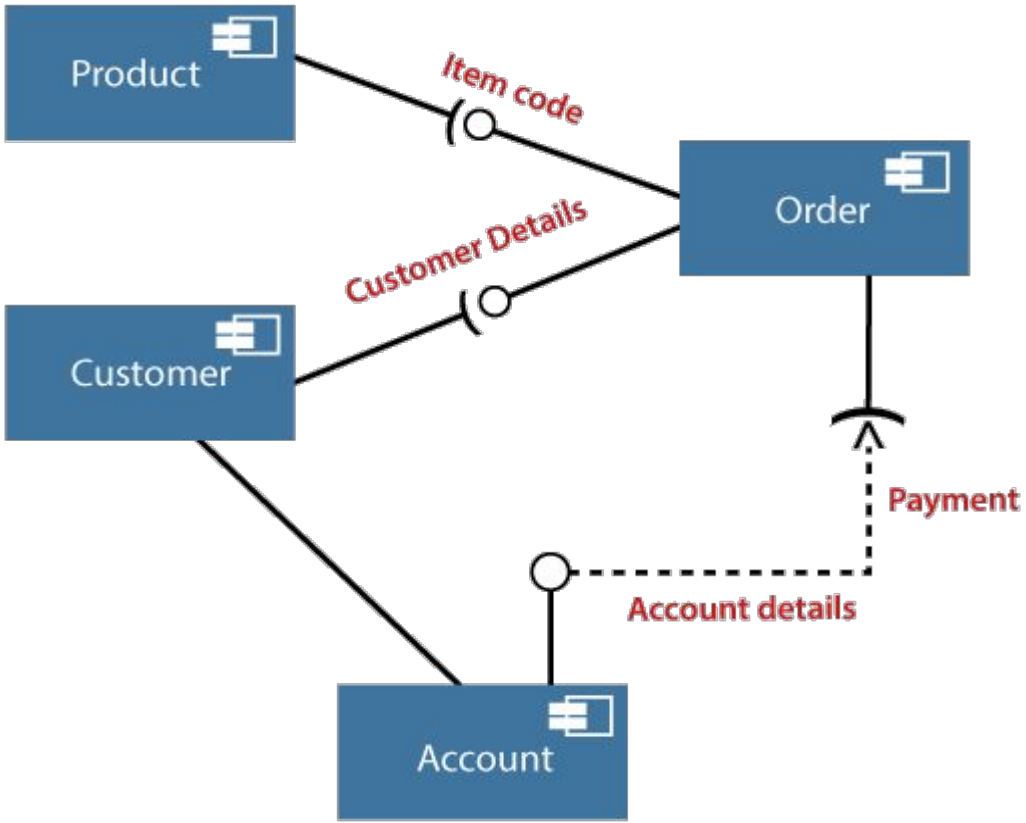
Use case

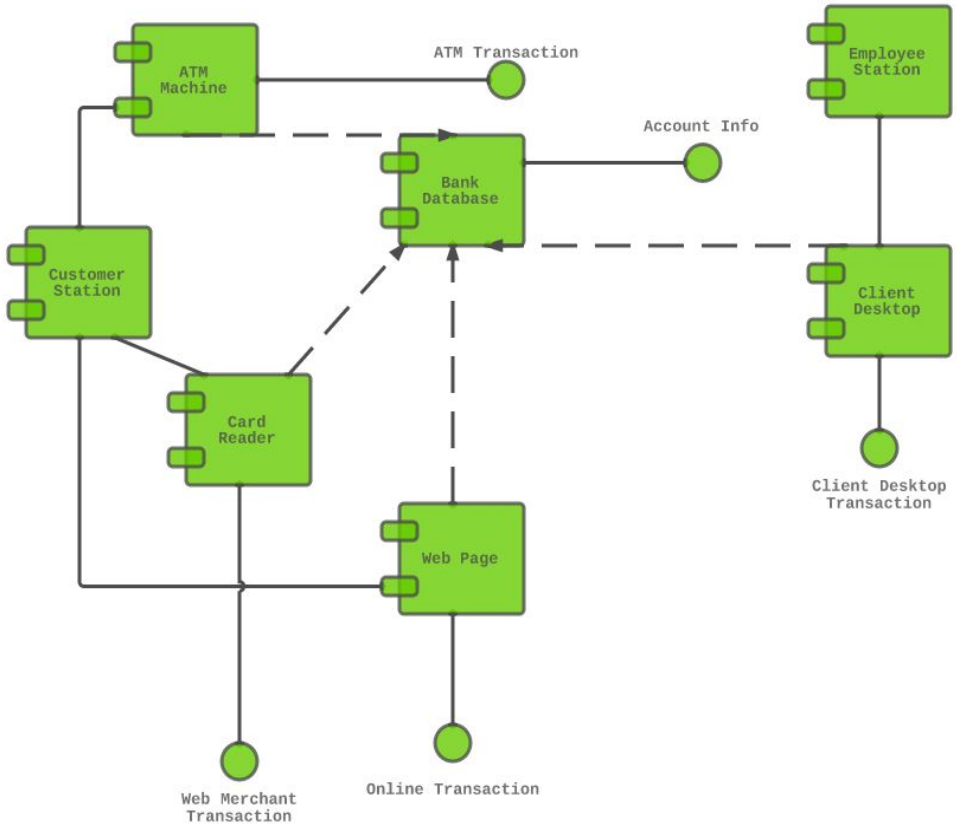
Use case is used to capture high level functionalities of a system.



component

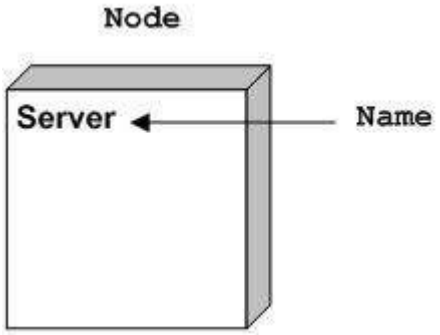
Component is used to represent any part of a system for which UML diagrams are made.





Node

Node is used to represent the physical part of a system such as the server, network, etc.

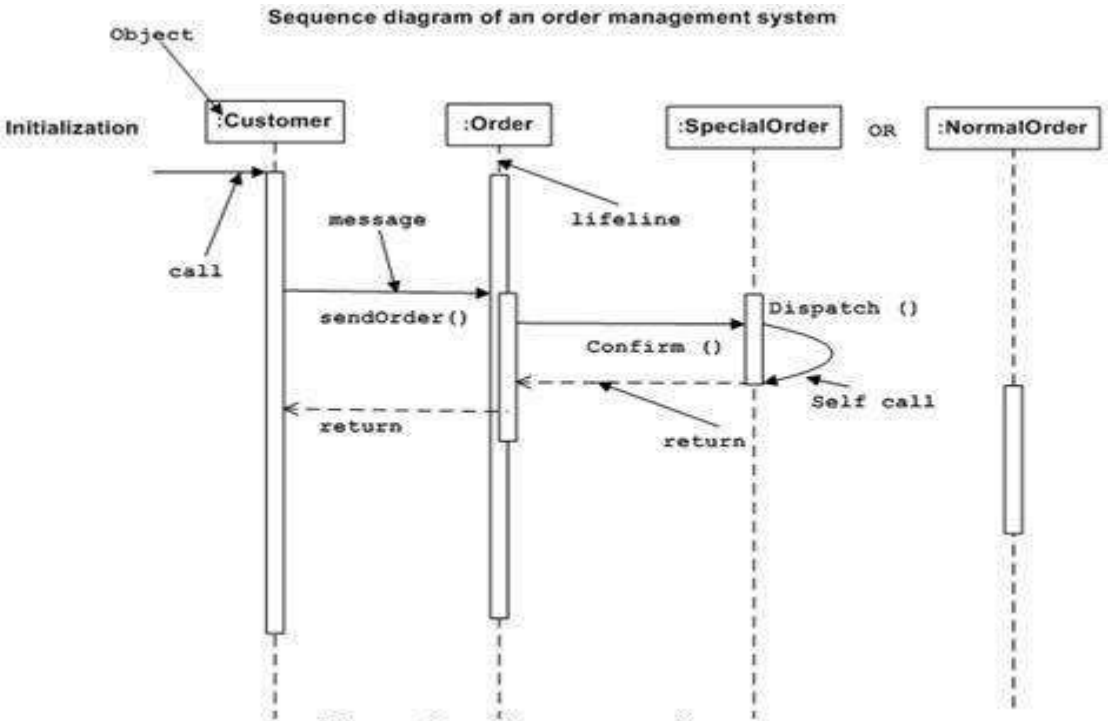


Behavioural Things

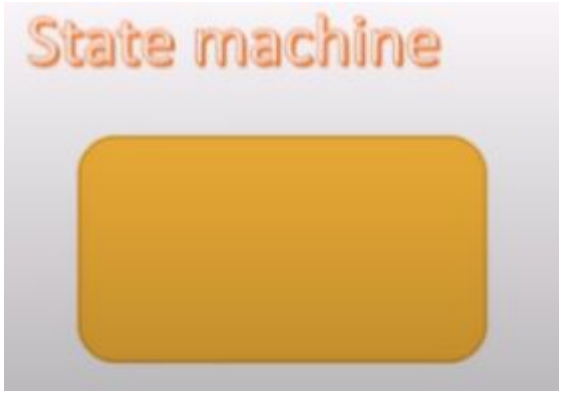
- A Behavioural Thing consists of the dynamic parts of UML models.
- 1. Interaction: Interaction is used to represent the communication among the components of a system.
- 2. State Machine: State machine describes the different states of a component in its life cycle.

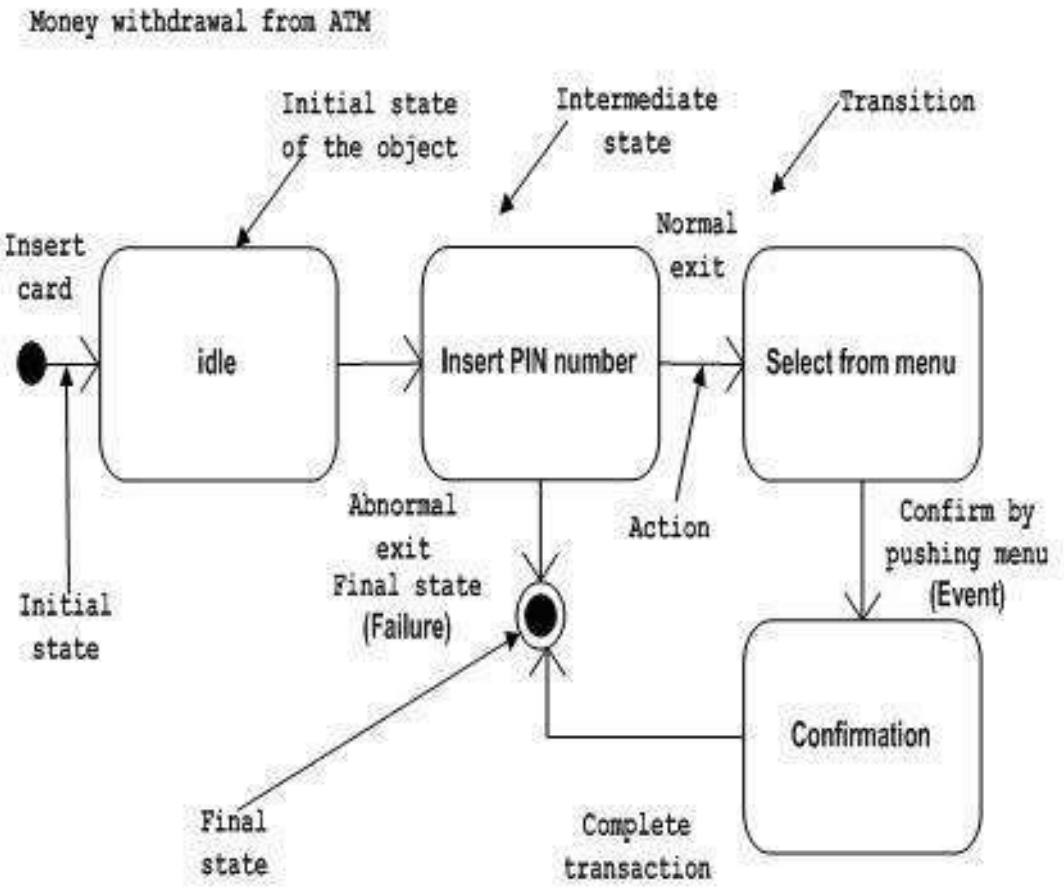
Interaction is used to represent the communication among the components of a system.





The state can be active, idle, or any other depending upon the situation.

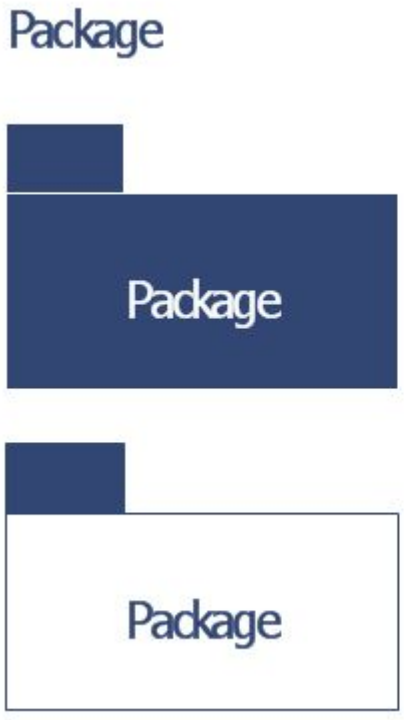


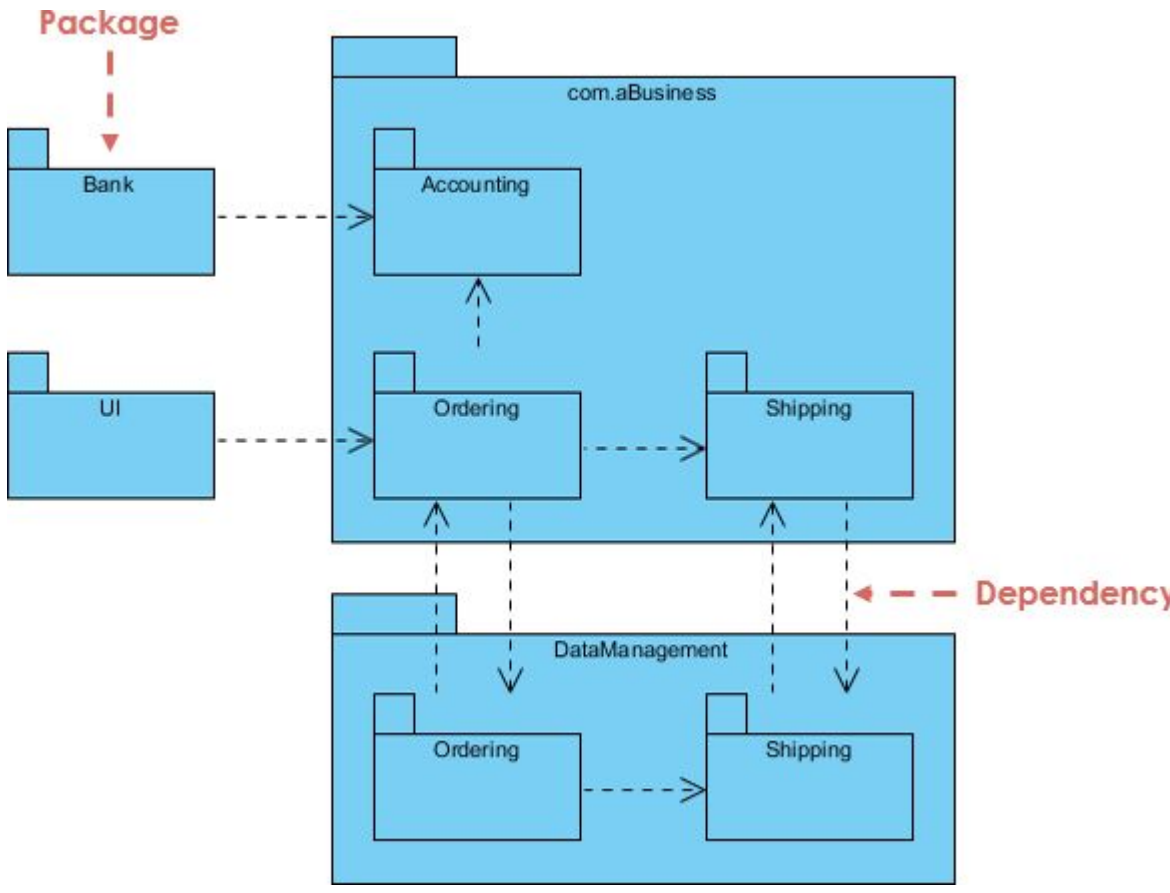


Grouping Things

- A mechanism to group elements of a UML model together.
- Only one grouping thing available for gathering structural and behavioural things.
- A general purpose mechanism for organizing elements into groups.

Package


Package
Package
Package

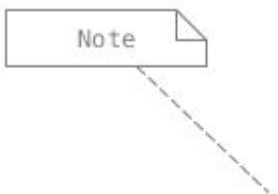

Package
com.aBusiness
Bank
Accounting
UI
Ordering
Shipping
Dependency
DataManagement
Ordering
Shipping

Annotational things

- A mechanism to capture remarks, and comments of Uml model elements.
- Note: It is the only one additional thing available. A note is used to render comments, constraints, etc of an UML element.

Note

Examples



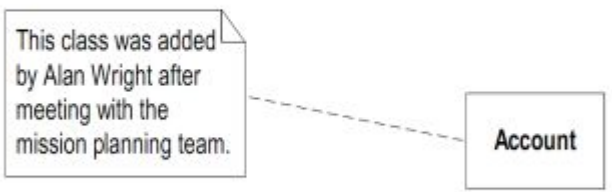
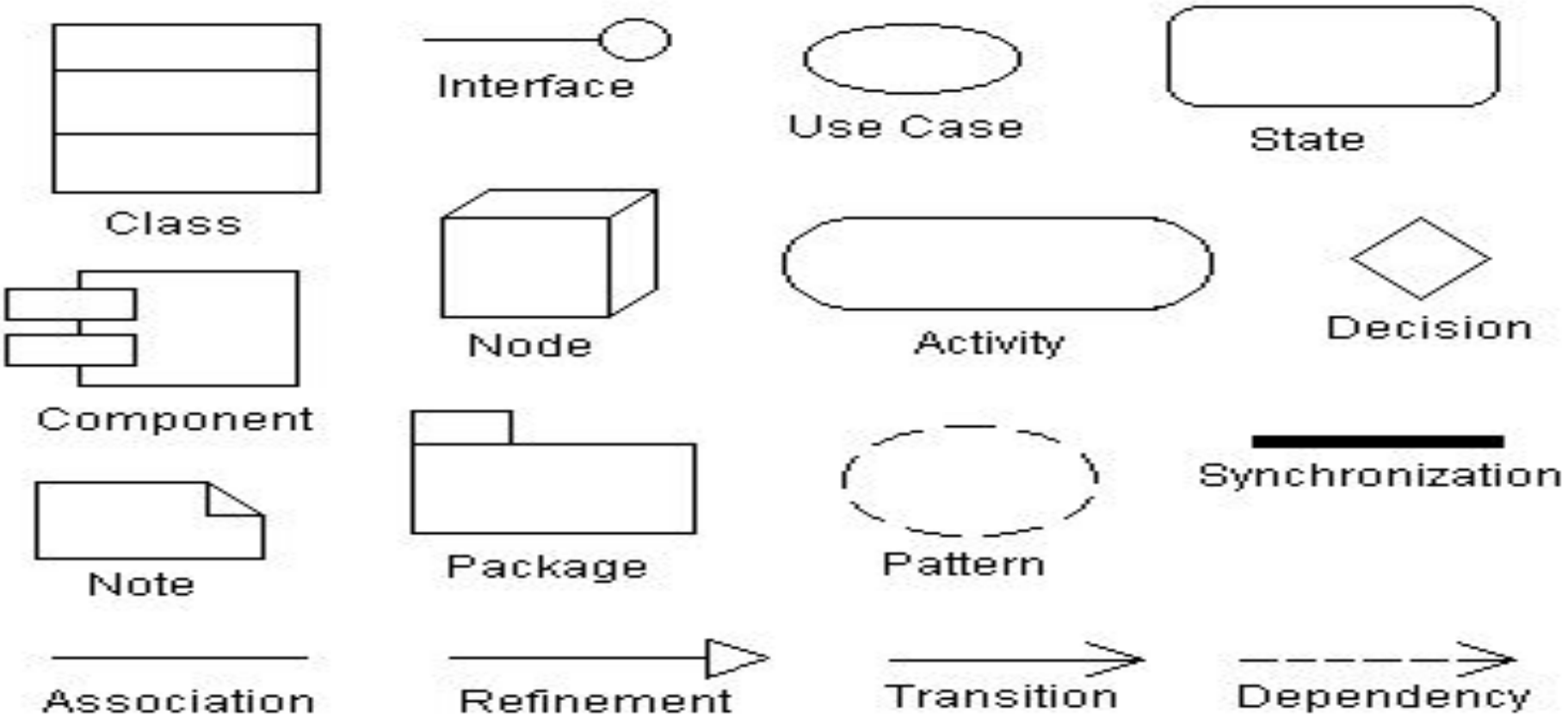
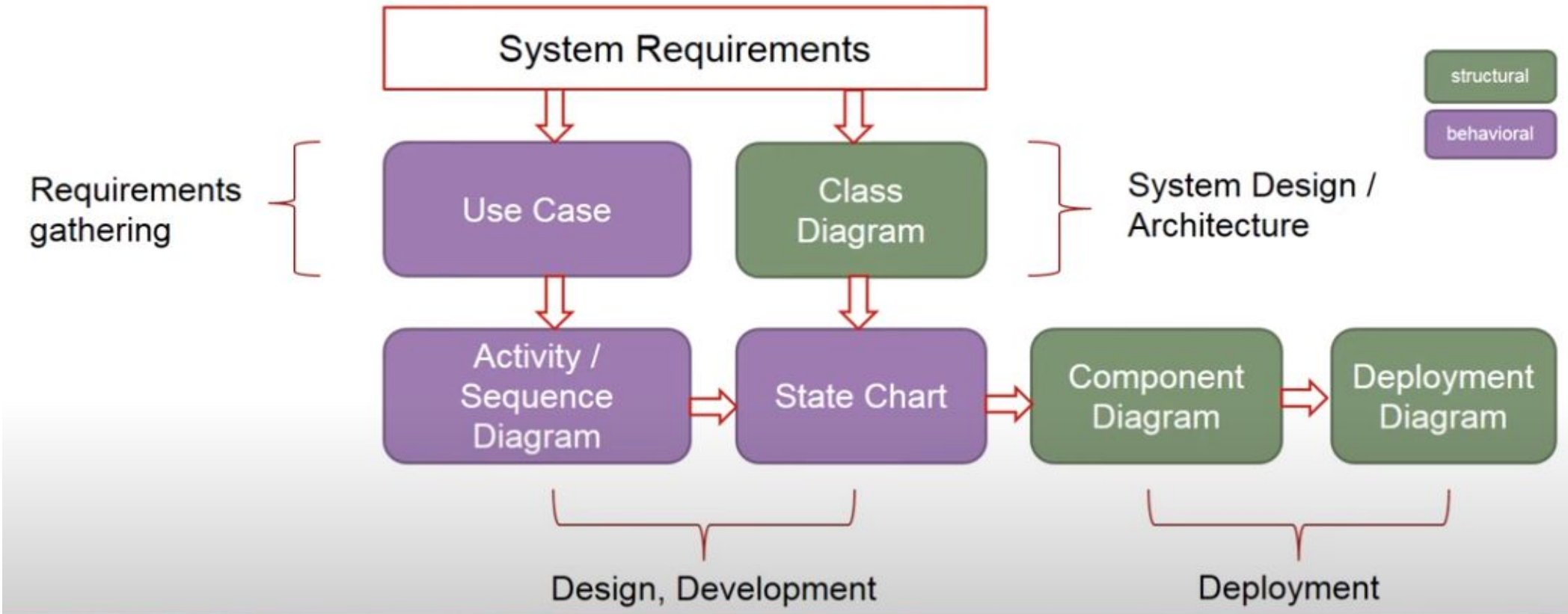




Figure 9.11 - Comment notation


Class
Interface
Use Case
State
Node
Activity
Decision
Component
Note
Package
Pattern
Synchronization
Association
Refinement
Transition
Dependency

DESIGN PROCESS



Use Case diagrams

- Cases of the usages by the user of the system
- We can have multiple use cases in a system

ACTORS

WHO INTERACTS WITH THE SYSTEM



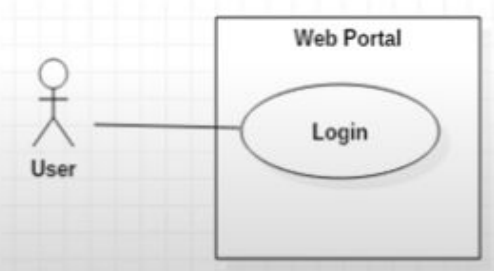
USE CASE

FUNCTIONALITY OR SERVICES PROVIDED BY THE SYSTEM



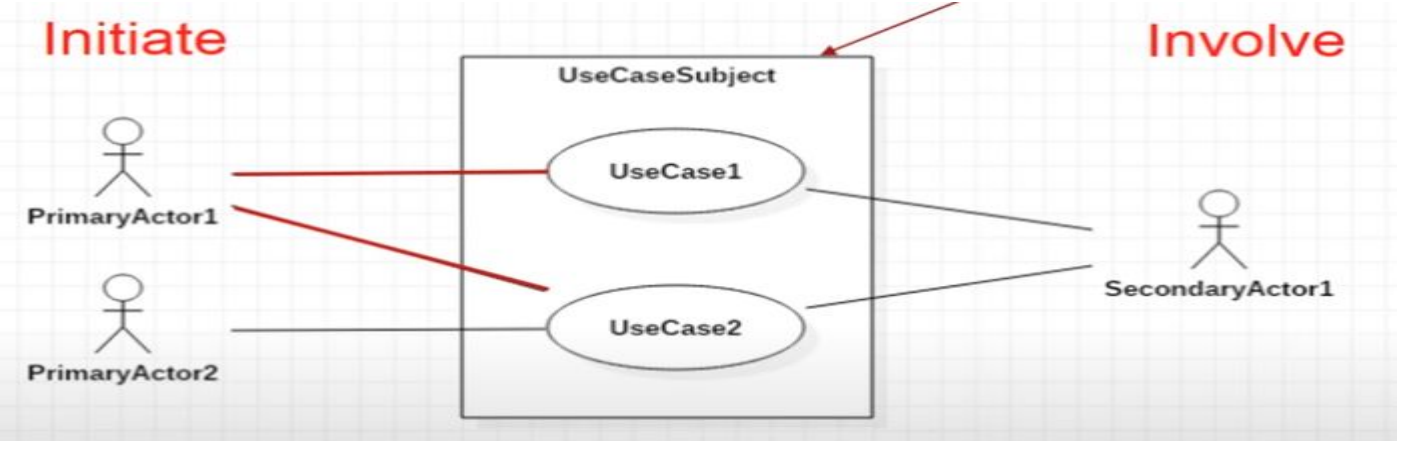
RELATION

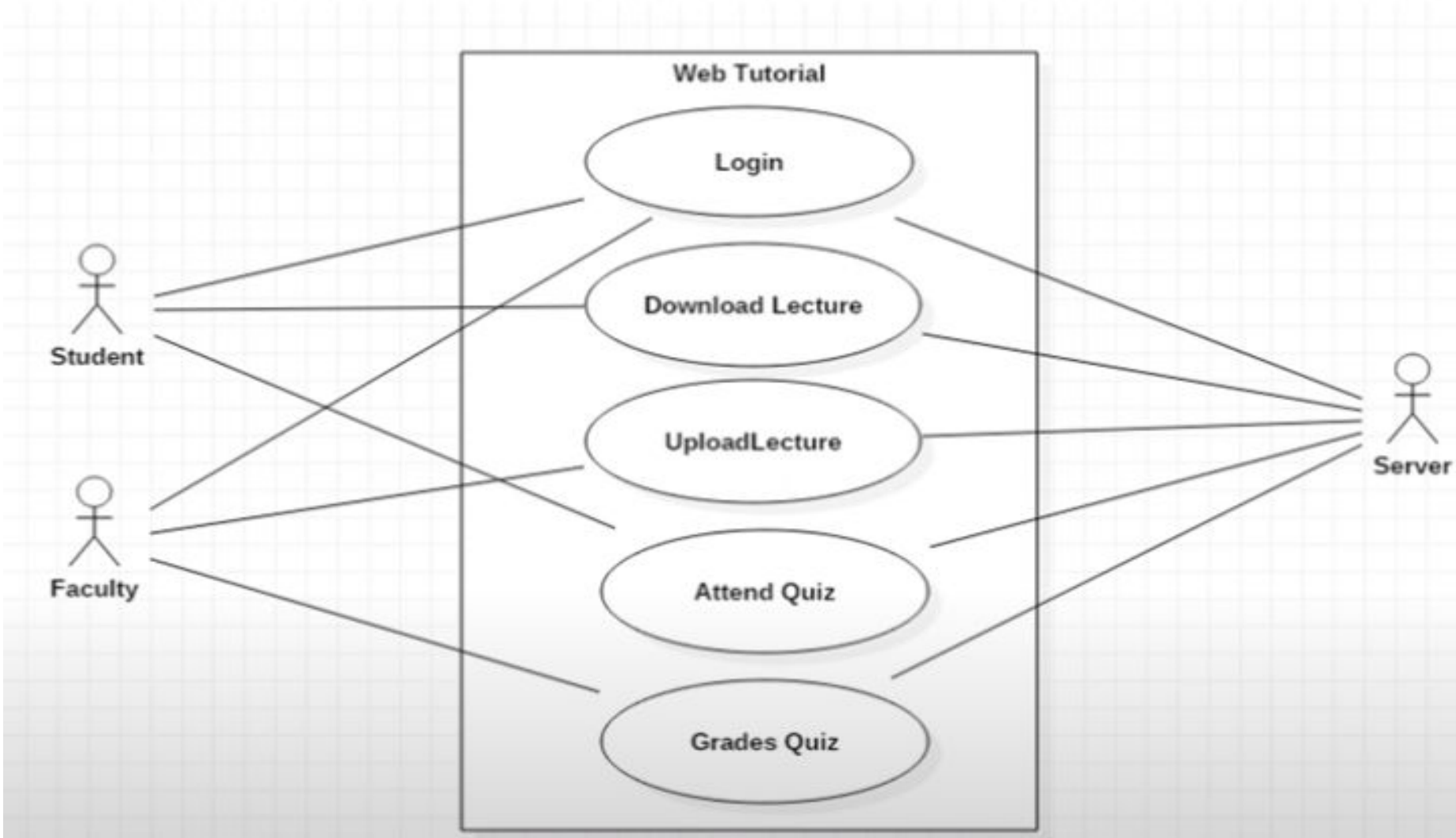
RELATION BETWEEN ACTORS AND THE SYSTEM



Simple rules

- Primary Actor: who can initiate the use case (always be left to the system)
- Secondary Actor: who can have involvement in a use case. They can not initiate the use case.((always be right to the system))



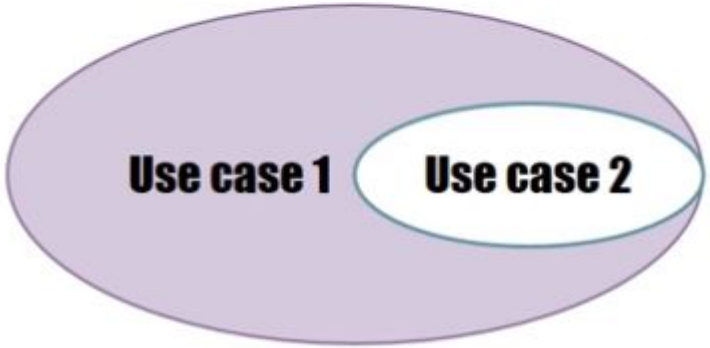

Web Tutorial
Login
Download Lecture
UploadLecture
Attend Quiz
Grades Quiz
Student
Faculty
Server

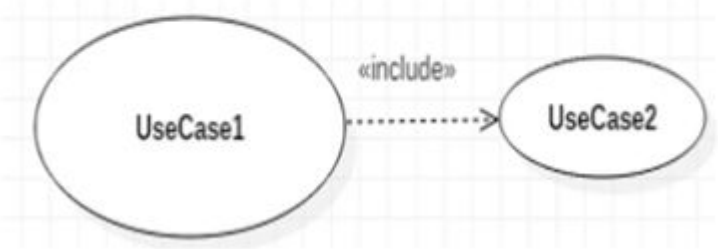
More components of use case

- One use case can be connected to another use case by **extend** or **include** relation.

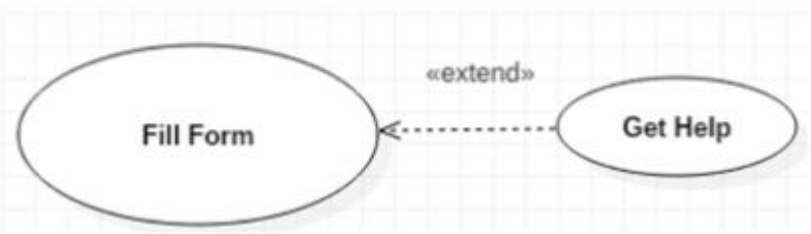


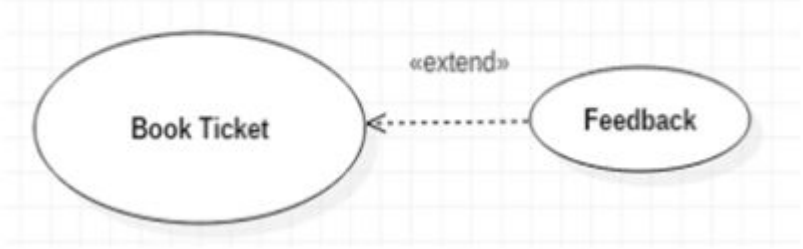
Include





Extend

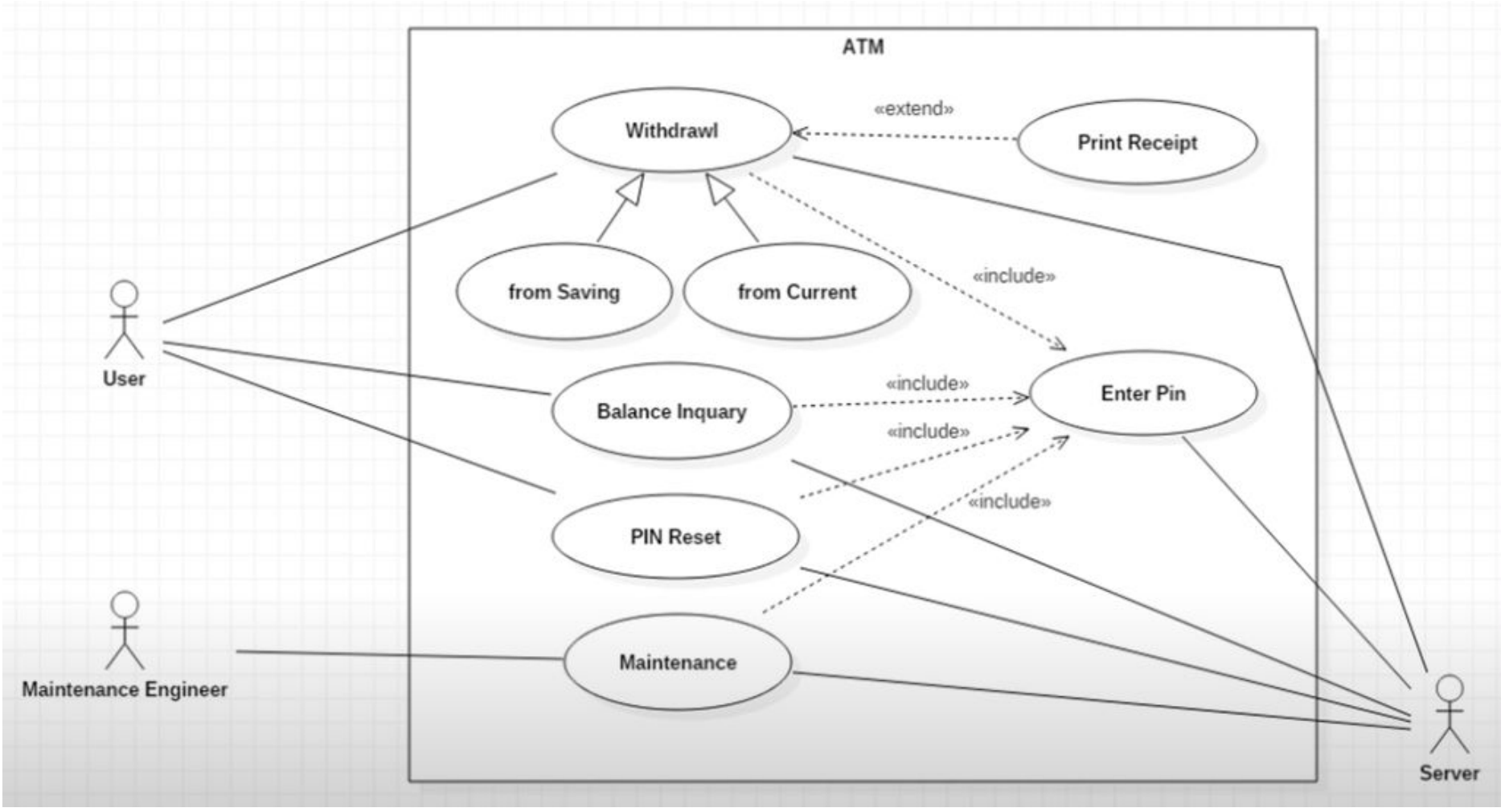




Use case Diagram for ATM Machine

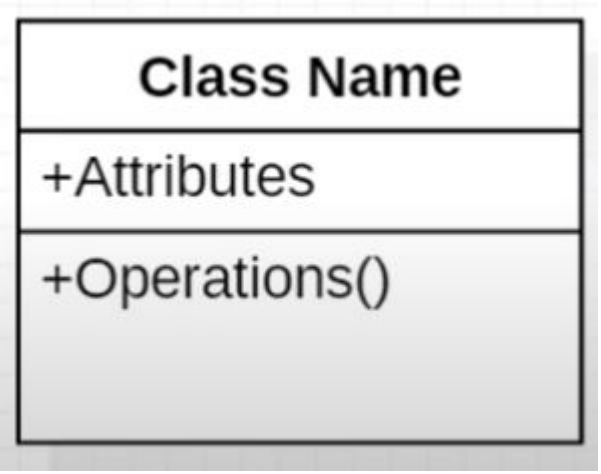
- Someone can:
- Withdrawal the amount
- Check balance
- Reset Pin
- Maintenance

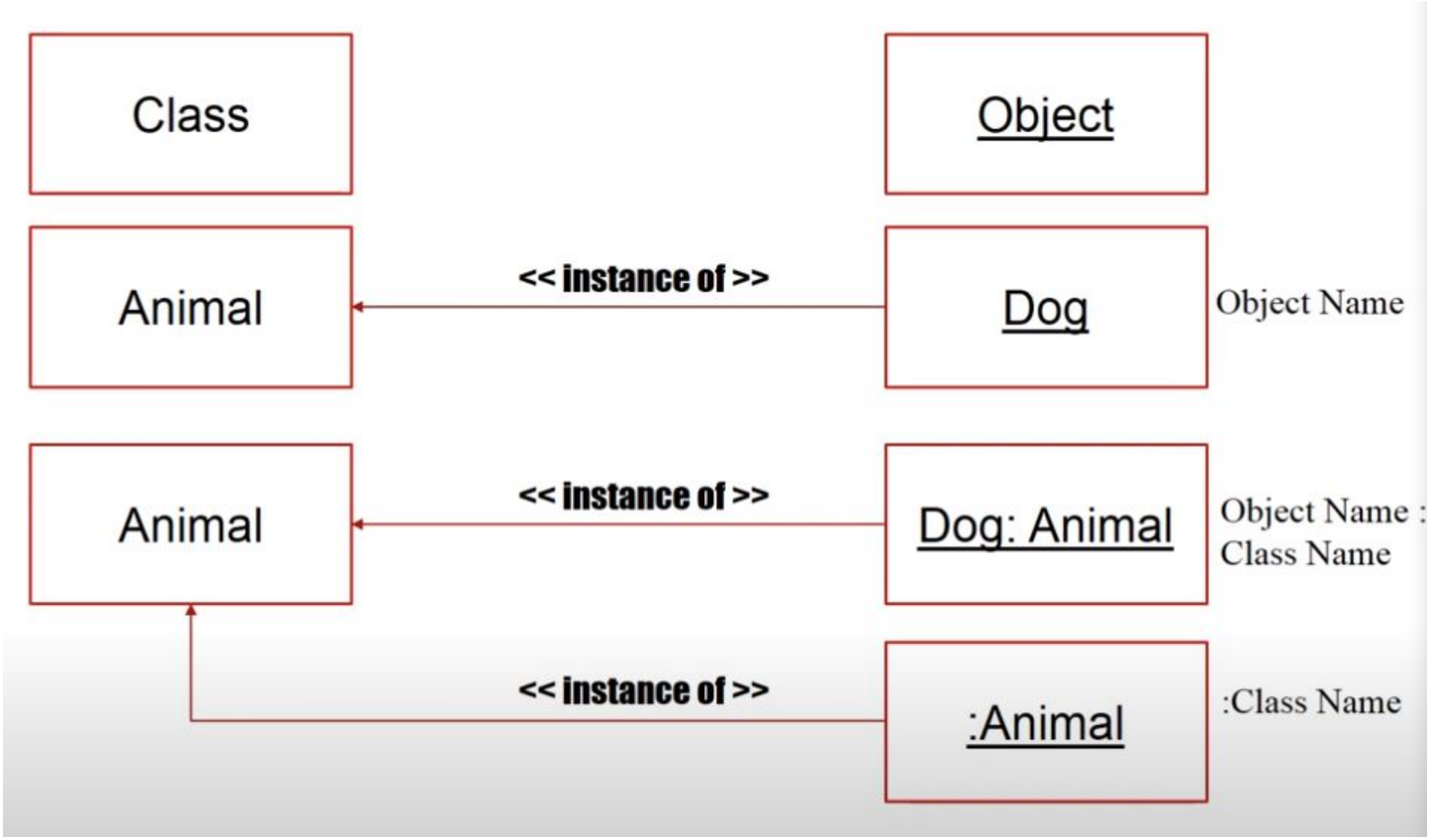
ATM

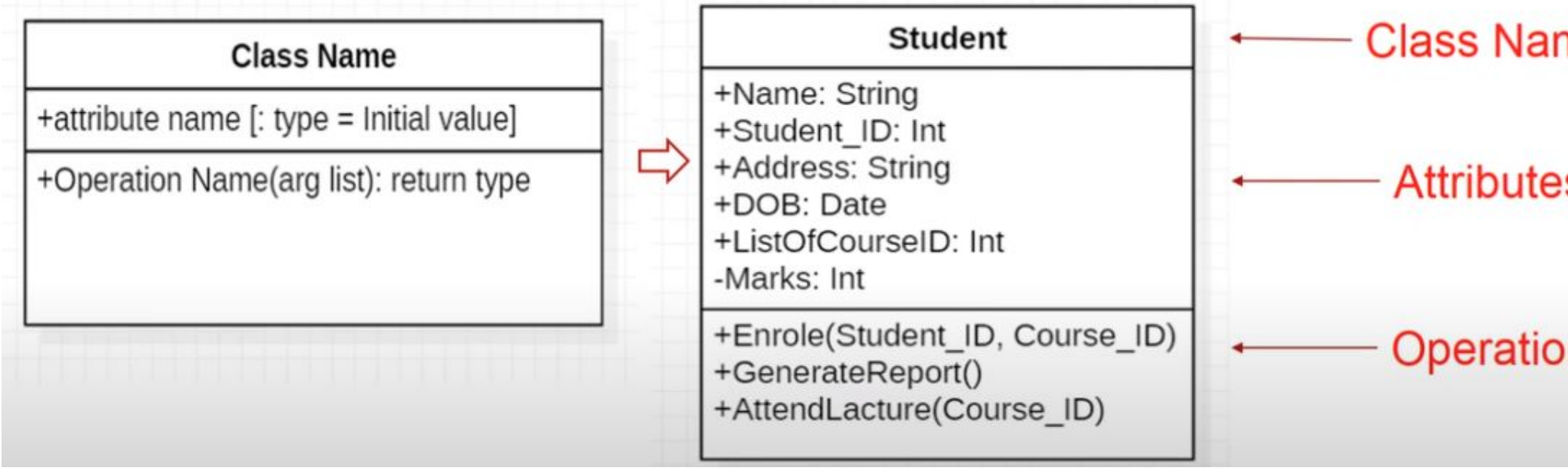

ATM
Withdrawl
«extend»
Print Receipt
from Saving
from Current
«include»
User
Balance Inquary
«include»
Enter Pin
«include»
PIN Reset
«include»
Maintenance Engineer
Maintenance
Server

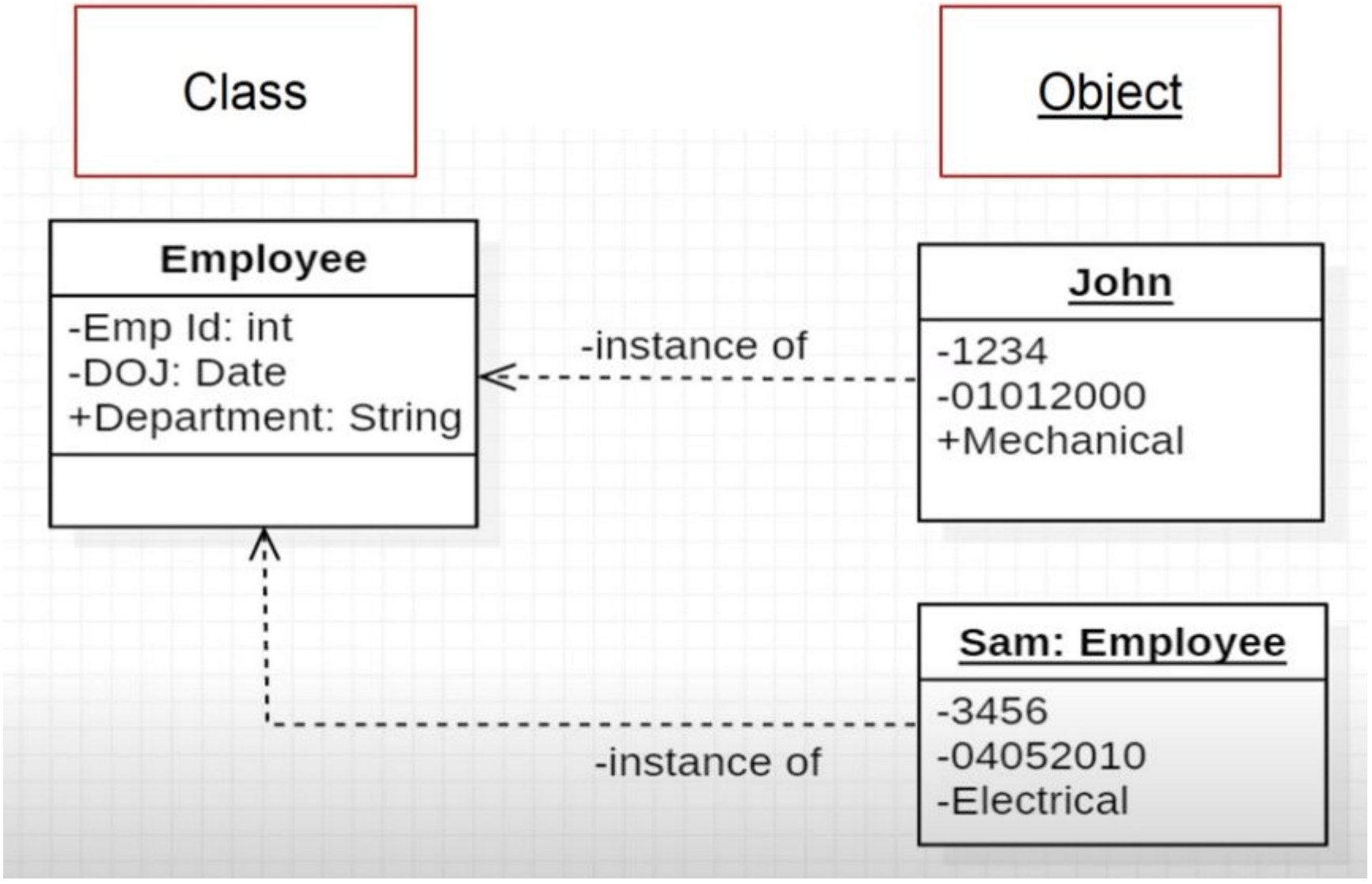
Class Diagrams with examples

- **Class:** It is a template or format. It represents entities with common characteristics or features.
- Features include:
  - Attributes-Data
  - Operations-Methods
  - Associations
-



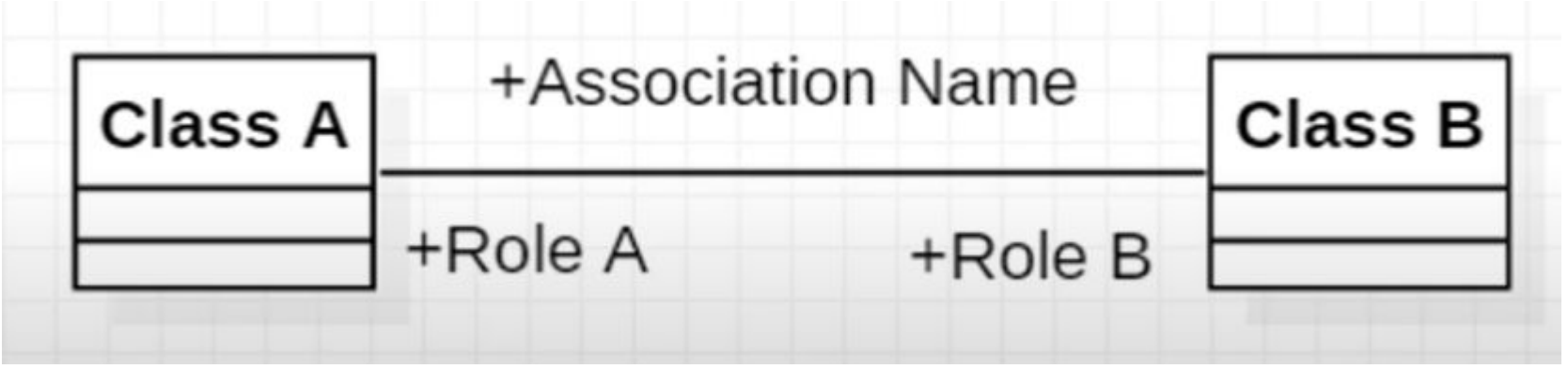

Class
Object
Animal
<< instance of >>
Dog
Object Name
Animal
<< instance of >>
Dog: Animal
Object Name :
Class Name
<< instance of >>
:Animal
:Class Name


Class Name
+attribute name [: type = Initial value]
+Operation Name(arg list): return type
Student
+Name: String
+Student_ID: Int
+Address: String
+DOB: Date
+ListOfCourseID: Int
-Marks: Int
+Enrole(Student_ID, Course_ID)
+GenerateReport()
+AttendLacture(Course_ID)
Class Nam
Attribute
Operatio

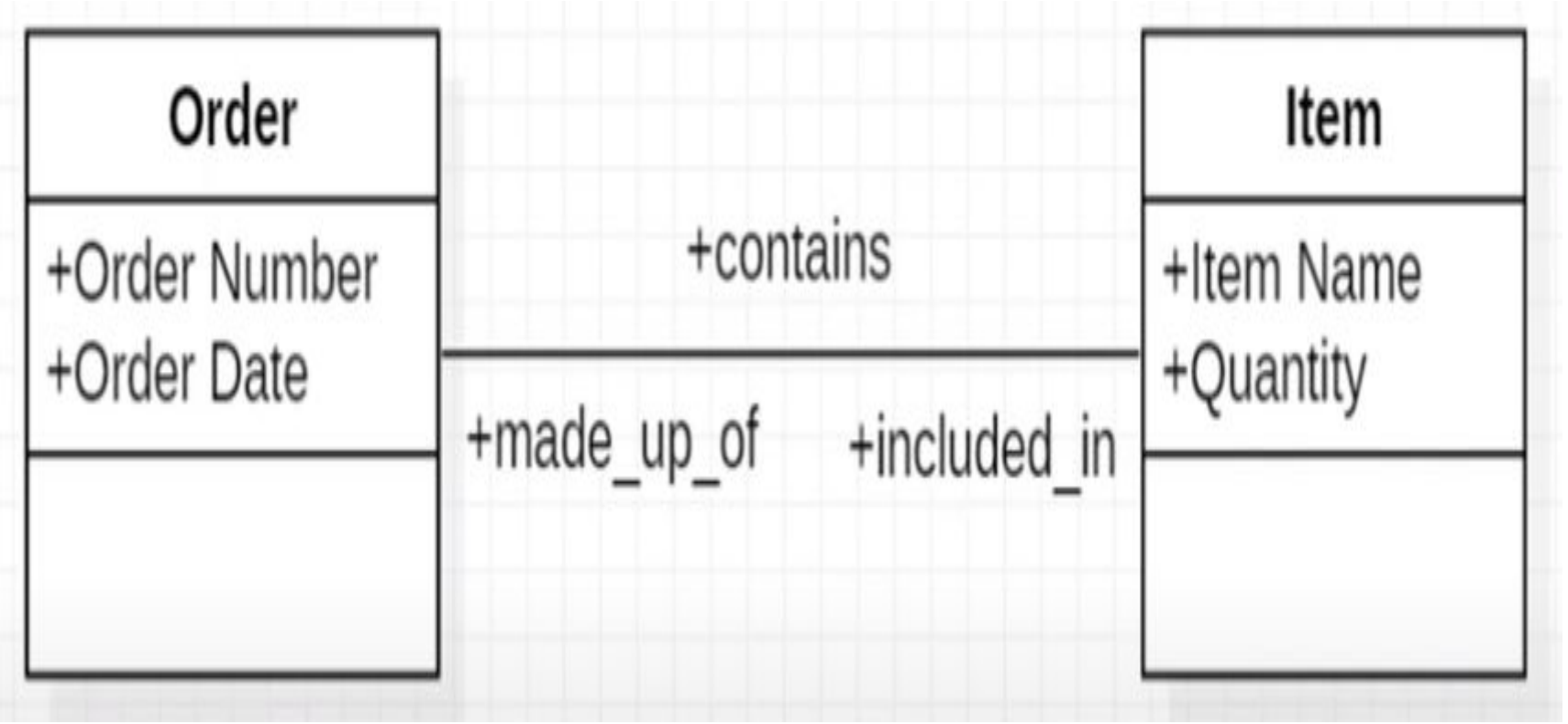

Class
Object
Employee
-Emp Id: int
-DOJ: Date
+Department: String
-instance of
John
-1234
-01012000
+Mechanical
Sam: Employee
-3456
-04052010
-Electrical
-instance of

How to connect classes with each other in some relationship

- **Association:**
- Structural relationship between peer classes
- Association can have a name and direction, or bi-directional.



Association Example



Multiplicity/Order

